
Cube DEX — Follow-up Security Review

Second Round

Serokell OÜ

9 September 2026



Cube DEX (aka Coffe) — Follow-up Security Review (Second Round)

1. Introduction

This is a follow-up to Smart Contract Code Review and Security Analysis Report for Cube DEX (the “original report”). The Contract author produced a fixed codebase, together with `RESPONSE-T0-AUDITORS.md`, which numbers each deviation from the report’s recommendations (`A1`, `A2`, ...) and each issue found by their own team (`SF-1`, `SF-2`, ...).

This round checks whether each original finding is actually closed by its fix, and whether the fixed codebase introduces or leaves open any new security-relevant issue. Severity definitions, the threat model, and the economic-model assumption are unchanged from the original report and are not repeated here.

Reviewed revision: commit `6f64097c60630676e7fa779b460e8237dd7ac533`, above the initially audited revision `070a55d`. General Token-2022 support, TransferFeeConfig extension accounting (SF-5), and the supervisor-role change (SF-1), added elsewhere, were left out of this revision by agreement and are not reviewed.

Method: every commit in the reviewed range was audited directly, then matched to its finding via the commit-prefix convention described in `RESPONSE-T0-AUDITORS.md` to check whether it actually closes that finding.

2. Status of original findings

ID	Finding	Status	Notes
H-01	Protocol-fee bypass via unauthenticated config creation	✓ Closes the finding	Real signer gate added; zero-fee floor case is also closed
H-02	Missing caller gate masked by a redundant argument	✓ Closes the finding	Same commit as H-01
H-03	First-deposit front-run leaves the pool permanently over-leveraged	✓ Closes the finding	Pool-admin gate on seed deposit + one-blockhash tx bundling
H-04	Sell-off cap measured against a snapshot the attacker controls	✎ Closes the finding partially — see L-14	The attacker-controlled-inflation path is closed; <code>range_manager_update</code> and swap’s buy side leave a narrower, Low-severity availability residual, see L-14

ID	Finding	Status	Notes
H-05	Banned mint extensions are a default, not an invariant	◆ Not closed by design (client decision)	Hard floor is <code>NonTransferable</code> only; <code>PermanentDelegate</code> / <code>TransferHook</code> stay overridable (A1); trade-off questioned at H-09 ; same override also admits untransferable mints, see L-17
H-06	Single-token deposits subtract protocol fees twice	✓ Closes the finding	Same pattern recurred at a second site, closed by the Contract author (SF-15)
H-07	Migration instruction silently re-enables every disabled token	✓ Closes the finding (residual accepted)	Opt-in flag added; replay residual accepted by owner (A12)
H-08	Actual/virtual skew breaks <code>single-token-liquidity</code> 's deposit split	⚠ Does not fully close — reopened as M-07	Clamp only covers the <code>actual > virtual</code> branch
M-01	Overflow checks disabled in the release profile	✓ Closes the finding	<code>overflow-checks = true</code>
M-02	Range-manager envelope caps velocity, not distance	📝 Closes the finding — with a note, see I-10	Rest of the fix came with SF-20
M-03	Upgrade-authority transfer and program freeze cannot execute	✓ Closes the finding	
M-04	Lopsided deposits decouple visible depth from priced depth	⚠ Closes the finding — opens a new issue, see M-08	<code>token_amounts</code> is now a ceiling; the fix's own rounding introduces a distinct gap, see M-08 ; also leaves <code>std</code> 's own pre-cap redundant, see L-16
M-05	Splitting a swap avoids the surge fee	📝 Closes the finding — with a note, see I-12	Worst-case spread down to 1.086×; narrow residual at I-12
M-06	<code>range_manager_update</code> 's "set" semantics create a lost-update race	✓ Closes the finding	Compare-and-swap guard; breaking argument-layout change
L-01	Range-manager updates ignore the pool kill switches	✓ Closes the finding	
L-02	One error code for three unrelated failures	✓ Closes the finding	Extended to remaining call sites by the Contract author (SF-6)
L-03	Constraint order makes an error branch unreachable	✓ Closes the finding	

ID	Finding	Status	Notes
L-04	Deployment verification never checks the admin it promises to check	✓ Closes the finding	Related exit-code bug fixed by the Contract author (SF-21)
L-05	Treasury initialization is unauthenticated	✓ Closes the finding	Gated on the program's own upgrade authority (A7)
L-06	Unchecked decimal-normalization overflow in the invariant calculation	✓ Closes the finding	
L-07	<code>withdraw_sol</code> never checks <code>source</code> belongs to the config	✓ Closes the finding	
L-08	<code>single-token-liquidity</code> 's SOL-withdrawal guard can never be satisfied	✓ Closes the finding	
L-09	Protocol-admin rotation never re-checks the new admin is the Treasury PDA	◆ Not closed by design (client decision)	Destination left unrestricted on purpose (A2)
L-10	<code>debug_withdraw_liquidity</code> performs an unchecked CPI	✓ Closes the finding	
L-11	Deflating virtual balance at a window rotation locks out honest sells	✎ Closes the finding partially — see L-14	Same root cause and fix as H-04
L-12	No working operator tool for the migration instruction	✓ Closes the finding	
L-13	Single-token deposits revert past a fixed-point ceiling	✓ Closes the finding	
I-01	Over-supplied deposit amounts are donated with no refund	✓ Closes the finding	Same fix as M-04
I-02	No minimum-output guard on subsequent deposits	✓ Closes the finding	Distinct error code <code>DepositTooSmall</code> (A9)
I-03	Swap event breaks the append-only field convention	✓ Closes the finding	
I-04	Sell-off cap and surge fee are inactive by default	◆ Not closed by design (client decision)	Defaults kept at zero (A10)
I-05	Freeze events omit the previous state	✓ Closes the finding	

ID	Finding	Status	Notes
I-06	Treasury-signed arbitrary CPI (present on mainnet, fixed in <code>main</code>)	— N/A	Was present in V4, already fixed in <code>main</code>
I-07	Single-token-deposit event sums raw amounts across mints of different decimals	✓ Closes the finding	Breaking wire-format change, accepted (A11)
S-01	Just-in-time liquidity can skim swap fees	■ No fix proposed — unchanged	Accepted design property
S-02	Withdrawing liquidity amplifies price impact	■ No fix proposed — unchanged	Accepted design property

Summary: most findings close cleanly. A few remain only partially closed and are reopened below; a couple are left open by deliberate client decision rather than an incomplete fix.

3. New findings

New findings, numbered continuing the original report's scheme. L-14, L-15, and M-07 trace back to an original finding whose fix doesn't fully close the underlying mechanism; H-09 revisits the trade-off behind an accepted residual risk (H-05); the rest are new issues surfaced while reviewing the fixed codebase.

Severity	Count
High	1
Medium	2
Low	4
Informational	5
Total	12

Verdict. The fixed revision is fit for deployment once H-09 is addressed, two Medium findings are either fixed or explicitly accepted. The remaining Low and Informational items do not block deployment.

High severity

H-09 — H-05's minimal-floor justification does not hold for the reviewed revision

Location: `cubic-pool/src/constants.rs:217` (`HARD_BANNED_EXTENSIONS = 1 << 9`)

Status: Revisits H-05 (not closed by design, client decision). The `PermanentDeLegate` drain itself is an accepted, documented residual risk (A1) and is not re-reported; the finding is that the trade-off behind that acceptance does not apply to the reviewed revision.

Reproduction: `scripts/audit-report/poc.sh` H-09

The response to the first report accepts the risk because a wider floor “excludes essentially the entire regulated-asset segment: PYUSD, USDG, AUDS, PAXG, tokenised equities and the Ondo family”. Those assets carry `TransferFee`, `TransferHook` or `Pausable`, and Token-2022 refuses the plain `transfer` instruction for accounts of such mints (`MintRequiredForTransfer`, `0x1f`); moving them takes `transfer_checked` plus fee- and hook-aware accounting, which is Token-2022 feature work added separately and not part of the reviewed revision. As reviewed, every pool transfer is a plain `token_interface::transfer` — seven call sites across `cubic-pool` and the single-token-liquidity helper (the zap).

So a PYUSD-class mint listed under a `Some(0)` override, which clears every default ban, cannot be deposited at all: `add_liquidity` reverts with `0x1f`. The one profile that both trades and reaches vault custody is a bare `PermanentDelegate` mint, exactly the H-05 attacker’s. Hard-banning issuer-power extensions therefore delists nothing the pool can trade as reviewed; on a scan of the top-100 Token-2022 mints by market capitalisation it removes only the two that are both tradeable here and `PermanentDelegate`-bearing, JAAA and CURR.

Impact: the H-05 vault drain stays open on a trade-off that does not apply to the reviewed revision. The mitigation priced as delisting the regulated segment currently costs nothing, so the accepted-risk decision deserves a second look.

Recommendation: while full Token-2022 support is pending, seed:

```
HARD_BANNED_EXTENSIONS =  
    NonTransferable  
    | PermanentDelegate  
    | TransferHook  
    | TransferFee  
    | Pausable
```

When that support lands, governance lowers the floor via `set_banned_extensions`; the floor is on-chain state for exactly this reason.

Medium severity

M-07 — H-08 fix is incomplete: the zap’s clamp only covers the `actual > virtual` branch

Location: `single-token-liquidity/src/math.rs` (`compute_allocations`)

Status: Reopens H-08 (fixed only partially)

Reproduction: `scripts/audit-report/poc.sh M-07`

H-08’s clamp (`effective_ab = min(ab, vb)`) is a no-op whenever `ab == vb`, so the linear deposit split still diverges from the pool’s real concave curve for any deposit large relative to a leg’s balance, skewed or not. Reproduced against the real curve functions: a balanced 500,000-unit deposit on a 1,000,000/1,000,000 pool gets a naive BPT ratio of 0.2 versus ≈ 0.2247 curve-aware — an $\sim 11\%$ relative BPT-ratio shortfall that grows with deposit size relative to depth.

Impact: not money loss under the current codebase — given the M-04 fix, `add_liquidity` never takes more than the proportional share, so a bad split simply mints less BPT and the unused excess returns to the depositor as dust.

Recommendation: apparently, no trivial fix exists. Options:

1. Solve on-chain via a 1-D root-find (the equal-ratio optimum has one real unknown, closed-form per leg), at extra compute cost.
2. Accept an off-chain-computed split, validating only $\sum \text{alloc}_i = \text{amount_in}$ — cheap and exact, but shifts trust to the caller and needs a staleness guard.
3. Keep today's split but revert when the realized `ratio_min` falls too far below the pool's spot-rate ratio — a provable upper bound, since the curve is concave, and unlike `minimum_bpt_amount` it checks live pool state rather than the caller's own quote.

M-08 — `add_liquidity` floors the credited amount independently from the minted BPT, diluting a thin live leg's LP backing

Location: `cubic-pool/src/instructions/user/add_liquidity.rs` (subsequent-deposit branch, ~L262-303, the per-leg crop loop using `FixedPoint::mul_down`)

Status: New — introduced by the M-04/I-01/I-02 fix

Reproduction: `scripts/audit-report/poc.sh M-08`

Both the minted `bpt_amount` and every live leg's credited amount `credit_i` are computed as independent `floor()` operations on the same `ratio_min` (`math/cubic_math.rs:118-158`), against different bases (`old_supply` vs. each leg's own `actual_i`). This violates the codebase's own stated rounding rule (`fixed_point.rs`'s module doc: round down when the user receives, round up when the pool keeps). The credit step is “the pool keeps tokens” and per that rule should round up, but uses `mul_down` — in the depositor's favour rather than the pool's. No `mul_up` (ceiling) variant exists yet in `fixed_point.rs`. The sibling `virtual_increase` computation (`add_liquidity.rs:343`) has the same independent-floor shape.

Impact: value transfer from existing LPs to the depositor, redeemable later via `remove_liquidity`, bounded by one raw unit per leg per call, so economically negligible on any real pool.

Recommendation: add `FixedPoint::mul_up` (ceiling) and use it for `credit_i` only — this cancels cleanly against `bpt_amount`'s floor since the two are rounded in opposite directions off different bases. `virtual_increase` is not a simple copy of that fix: rounding it the same way reopens a worse, single-call divergence (~50% in one call, verified), and leaving it floored reopens a slower unbounded multi-call divergence (~16.5% over 20 rounds, verified). It needs deriving from `credit_i`'s realized per-leg ratio instead:

```
let realized_ratio_i = FixedPoint::div_up(credit_i as u128, actual_i as u128)?;
let virtual_increase_i = FixedPoint::mul_up(virt_i as u128, realized_ratio_i)?;
```

Low severity

L-14 — H-04/L-11 fix is incomplete: range-manager updates (and the swap's buy side) never rescale the sell-off window

Location: `cubic-pool/src/instructions/user/range_manager_update.rs:187`

Status: Reopens H-04/L-11, narrowly — fixed only partially

Reproduction: `scripts/audit-report/poc.sh` L-14

The H-04/L-11 fix keeps a pool's sell-off cap honest by rescaling its sliding window (`rescale_window_on_liquidity_change`) whenever `virtual_balance` changes through `add_liquidity` or `remove_liquidity`. `range_manager_update.rs` changes `virtual_balance` too, but writes it directly and never calls that rescale, so an already-open sell-off window keeps using the snapshot it captured before the update for the rest of that window, regardless of which way `virtual_balance` just moved.

When the range manager grows a token's virtual balance, the window's cap stays anchored to the smaller, pre-growth snapshot instead of tracking the token's real, now-larger size. For the remainder of that window the cap is stricter than `max_selloff_pct` actually allows for the pool's real current balance: a sell that would be entirely legal against that real balance can revert with `MaxSelloffExceeded`, purely because the range manager moved `virtual_balance` in between.

The swap's buy side has the same gap — moving `virtual_balance_out` (`swap.rs:359-361`) skips the rescale too — though no way was found to turn it into a profitable attack on its own.

Impact: legitimate sells can be wrongly rejected, or, below the revert threshold, charged a larger surge fee than the real balance justifies, for the rest of a window after any range-manager update that grows a token's virtual balance. This is an availability/UX issue rather than a fund-loss path, on pools with both `max-selloff` and a range manager enabled — both opt-in, off by default (I-04).

Recommendation: add selloff-window awareness (rescale, at minimum) to `range_manager_update.rs` — the primary fix. As defense in depth, also rescale the sold/bought token's balance on the swap's buy side.

L-15 — Disabling then re-enabling `max_selloff_pct` within one window can resume a stale, un-rescaled snapshot

Location: `cubic-pool/src/instructions/user/set_max_selloff.rs`

Status: New — related to the H-04/L-11 fix (L-14)

Reproduction: `scripts/audit-report/poc.sh` L-15

`rescale_window_on_liquidity_change` deliberately skips any token with `max_selloff_pct == 0` — its window state is meaningless until an admin opts in. But `set_max_selloff.rs` explicitly does not reset the window accumulators when the policy changes. Combine the two: a pool admin disables the cap, `add_liquidity` / `remove_liquidity` move `virtual_balance` one or more times while disabled (skipped, never rescaled), then the admin re-enables the cap before the stale window would have rotated on the wall clock. `check_and_advance` sees a nonzero snapshot and no rotation due, so it reuses the old, un-rescaled snapshot instead of re-capturing live `virtual_balance`.

Impact: shrinking `virtual_balance` while disabled leaves a stale, now-too-large snapshot in place, so re-enabling reuses a cap looser than the pool's real current balance justifies — surge fee reads an oversized sell as small. Reaching it needs a pool-admin config toggle rather than being purely attacker-driven, narrowing the blast radius. The mirror direction matches L-14's shape exactly: if `virtual_balance` grows instead while disabled, the same stale, now-too-small snapshot is reused as-is on re-enable, making the cap wrongly restrictive instead.

Recommendation: either reset (or force-rotate) the affected tokens' window state in `set_max_selloff.rs` when `max_selloff_pct` transitions between `0` and nonzero, or have `rescale_window_on_liquidity_change` track liquidity movement even while disabled.

L-16 — stld still pre-caps deposits to the LP ratio itself, redundant with (and lossier than) the M-04 fix

Location: `single-token-liquidity/src/instructions/user/deposit_single_token.rs:293-305`

Status: New, caused by the M-04/I-01 fix

`add_liquidity`'s M-04 fix now treats `token_amounts` as a ceiling and does its own `ratio_min` crop internally, leaving any excess untouched in the caller's account. `deposit_single_token.rs` still runs its own client-side ratio-min pass (`cap_deposit_amounts_to_lp_ratio`) first, whose doc comment still describes the pre-fix “donation” model as the reason it exists.

Impact: none reachable — the direction is conservative (never over-mints) and the shortfall is refunded as dust, not lost. Purely avoidable precision loss and needless compute on every zap deposit.

Recommendation: drop `cap_deposit_amounts_to_lp_ratio`, and don't update `deposit_amounts`, in `deposit_single_token.rs:305`.

L-17 — Pool admission accepts mints the pool cannot transfer, so the pool bricks on its first deposit

Location: `cubic-pool/src/util/token.rs (check_mint_extensions)`

Status: New — surfaced reviewing the H-05 fix; same transfer path as H-09

Reproduction: `scripts/audit-report/poc.sh L-17` (the first test of H-09's proof: a fee-bearing mint passes admission, the first `add_liquidity` reverts)

`check_mint_extensions`, called from `initialize_cubic_pool.rs`, rejects only what the effective bitmap bans plus its unconditional value checks (an armed hook, a frozen default account state, an unknown extension type, the native mint). `TransferFeeConfig`, an inert `TransferHook` and `Pausable` live in the overridable `DEFAULT_BANNED_EXTENSIONS`, so a creator override that clears them admits such a mint — although, as reviewed, every pool transfer is a plain `token_interface::transfer`, which Token-2022 refuses for accounts of these mints. The pool is created successfully and its first `add_liquidity` reverts with `MintRequiredForTransfer`, leaving the creator's rent in a pool nobody can use.

The single-token-liquidity helper has no admission check of its own either: its

`TokenExtensionsUnsupported` error variant (`single-token-liquidity/src/errors.rs:179`) is declared but never raised.

Impact: no fund loss; a wasted pool creation, with the creator's rent stranded, until full Token-2022 support is added.

Recommendation: reject `TransferFeeConfig`, `TransferHook` and `Pausable` at pool creation regardless of the override (H-09's wider hard floor does this), and lift the check together with the Token-2022 support.

Informational

I-08 — Pool-creation PDA is front-runnable / squatable

Location: `cubic-pool/src/instructions/user/initialize_cubic_pool.rs:29-35`

Status: New — missed in the original review

Reproduction: `scripts/audit-report/poc.sh I-08`

The pool PDA is seeded only by `[CUBIC_POOL_SEED, config.key(), pool_id]` with no caller gate: whichever signer's transaction lands first for a given `(config, pool_id)` becomes `pool_admin`, with attacker-chosen mints and extensions. Distinct from H-03 (front-running the first deposit into an already-created pool) — nothing addresses front-running creation itself. `pool_id` is not secret: deploy tooling hard-codes small sequential values or takes them as plain operator input, and neither checks whether the target PDA already exists before submitting.

Impact: at minimum griefing/denial of service (the legitimate creator's transaction fails and must retry); worse if an off-chain consumer trusts `(config, pool_id)` without independently checking the mints.

Recommendation: verify the pool's mints/extensions after creation, from the tooling side, rather than trusting `(config, pool_id)` alone.

I-09 — `PoolInitialized` event omits `pool_id`

Location: `cubic-pool/src/events.rs:44-52` (`PoolInitialized`)

Status: New — missed in the original review

`PoolInitialized` does not carry `pool_id`, so an indexer watching only logs cannot recover it from the event. The read-only `PoolInfo` event does include `pool_id` — an inconsistency between the two.

Impact: none beyond an off-chain indexing gap.

Recommendation: append `pool_id: u64` to `PoolInitialized` for consistency with `PoolInfo`.

I-10 — Organic swap flow can drift virtual/actual skew far enough to lock the M-02/SF-20 leverage band

Location: `cubic-pool/src/instructions/user/range_manager_update.rs`

Status: New — surfaces from the M-02/SF-20 fix

`swap.rs` moves `virtual_balance` and `actual_balance` by the same absolute amount, so ordinary one-directional trading over the pool's lifetime can drift the virtual/actual ratio arbitrarily far.

Once that drift exceeds what a single `max_vb_change_pct`-bounded step can correct, no legal range-manager update exists — every candidate value is either still outside the band or unreachable in one hop — leaving that token's `virtual_balance` stuck until opposite-direction flow drifts it back, or the pool admin intervenes.

Recommendation: none offered — this depends on the intention behind the M-02/SF-20 band and is noted for the Contract author's awareness rather than as a defect to fix unilaterally.

I-11 — `migrate_to_v5` trusts, rather than verifies, that no V4 pool ever set a nonzero sell-off cap

Location: `cubic-pool/src/instructions/user/migrate_to_v5.rs`

Status: New — pre-existing, out of scope for the original review's v4→v5 migration axis, now checked

Reproduction: `scripts/audit-report/poc.sh I-11` (cargo test only, no validator)

The handler's doc comment asserts every live V4 pool has `max_selloff = 0` and `max_selloff_period_length = 0`, and treats that as sufficient grounds to reinterpret those bytes as the new `max_selloff_pct`, `variable_fee_threshold_pct`, `variable_fee_slope_low_pct` and `variable_fee_slope_high_pct` fields, with no reset and no on-chain validation. V4 shipped a real, pool-admin-callable `set_max_selloff` instruction with nothing preventing a nonzero value, so “always zero” is an operational claim about mainnet history rather than something the chain enforces.

Impact: if migration is ever run over a pool that had a nonzero sell-off cap under V4, the new fields take on pseudo-arbitrary values, potentially breaking contract functionality for that pool.

Recommendation: before any V4→V5 upgrade, confirm via off-chain tooling (`pull-state / pool-info`) that every live pool's `max_selloff / max_selloff_period_length` are zero.

I-12 — Splitting a swap can dodge the surge fee again when the taxed part of the window is narrow

Location: `cubic-pool/src/math/surge_fee.rs` (`fill_ratio`)

Status: Reopens M-05, narrowly — surfaced reviewing the M-05/SF-16/SF-18/SF-20 fix

A swap's position in the sell-off window is rounded down to the nearest 0.01% step of the cap before the surge fee looks at it. If a swap's taxed range is narrow enough to always round into the same step, the fee treats it as one point and charges only the endpoint rate — M-05's original bug, reopened by rounding rather than by splitting. Triggered by any threshold/kink configured near 100% fill, a valid configuration.

Impact: reopens the split-swap fee dodge, worst exactly where the fee is meant to bite hardest — right before the sell-off cap. Requires the surge curve enabled (opt-in, see I-04) and the pool admin to have placed the threshold/kink near 100% fill; a normally-configured pool is not affected, hence informational rather than a direct vulnerability. Observed: one swap pays 1,860 at a threshold of 99.98% fill with a 2-step taxed span; split into 4, the same swap pays 1,396 — 1.33× cheaper, past the Contract author's own 1.1× bound from the M-05 fix.

Recommendation: make `fill_ratio` operate in the same `ONE` (1e18) scale instead of rounding to 10,000 buckets first, or require the configured threshold to stay far enough from 100% of the cap.

4. Disclaimers

Unchanged from the original report, Section 9.

Appendix A — Reproduction

`bash scripts/audit-report/poc.sh <id>` runs one proof, or all of them without an argument. On-chain proofs need a running `solana-test-validator`; setup is in `scripts/audit-report/README.md`.

ID	Finding	Type	Observed result
H-09	H-05's minimal-floor justification does not hold as reviewed	on-chain	a bare <code>PermanentDelegate</code> mint deposits fine; a <code>PYUSD</code> -class mint passes admission but <code>add_liquidity</code> reverts with <code>MintRequiredForTransfer (0x1f)</code>
M-07	Zap's actual/virtual clamp is a no-op when unskewed	on-chain	naive split mints ~11% less BPT than curve-aware for the same deposit
M-08	<code>add_liquidity</code> floors the credited amount independently from minted BPT	on-chain	attacker nets profit while the diluted leg's balance ends bit-for-bit unchanged
L-14	Range-manager (and swap buy side) never rescale the sell-off window	on-chain	a sell legal against the token's real (1.5x-grown) balance reverts with <code>MaxSelloffExceeded</code> against the stale snapshot
L-15	Disable/re-enable of <code>max_selloff_pct</code> resumes a stale snapshot	on-chain	stale snapshot lets a 5x-oversized sell through
L-17	Pool admission accepts a mint it cannot transfer, bricking on first deposit	on-chain	a <code>PYUSD</code> -class mint is admitted under a <code>Some(0)</code> override, then <code>add_liquidity</code> reverts with <code>MintRequiredForTransfer (0x1f)</code> on the first deposit
I-08	Pool-creation PDA is front-runnable / squattable	on-chain	attacker's tx lands first; legit deployer's tx for the same pair fails; attacker holds <code>pool_admin</code>
I-11	<code>migrate_to_v5</code> trusts rather than verifies V4's sell-off bytes were zero	cargo test	after migration, plausible V4 bytes decode into a config that reverts every swap with <code>MaxSelloffInvalidConfig</code> ; plus other scenarios